

第 24 章 挂载、fstab、Swap 与启动持久性

把“当前可用”和“下次启动仍成立”拆成两层状态，再用挂载关系、fstab、systemd unit 与 Swap 证据逐层验收。

对象模型操作语义验证诊断经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：目录存在、当前挂载、fstab 声明、systemd unit 与启动后状态不是同一件事；Swap 也要把签名、当前启用和持久配置分开证明。

- 01 识别源与目标确认文件系统或 Swap 对象是谁
- 02 读取当前状态用 `findmnt` 或 `swapon --show` 建立事实
- 03 修改当前关系 `mount`、`umount`、`swapon`、`swapoff`
- 04 写入持久策略 `fstab` 六字段、UUID、选项与优先级
- 05 理解启动 `unit` 生成 `.mount` / `.swap` 及失败证据
- 06 分层验收静态检查、重入测试、功能与重启边界

专题地图

- 知识专题挂载对象、当前关系与遮蔽
- 操作专题 `findmnt` 证据与当前挂载
- 诊断专题 `busy` 引用与安全卸载
- 知识专题 `fstab` 六字段、选项与启动风险
- 知识专题 `systemd mount unit` 与网络边界
- 操作专题 Swap 三层状态与优先级
- 诊断专题从症状定位到正确状态层
- 经典任务持久挂载与新增 Swap

阅读时持续回答

1. 对象是文件系统、挂载关系还是 Swap 区域？
2. 当前 `source` 与 `target` 分别是什么？
3. 这一条命令改变当前状态还是持久配置？
4. 当前选项与 `fstab` 文本是否一致？
5. 哪个证据能证明 `unit` 生成与启动结果？
6. `mount -a` 或 `swapon -a` 实际测试了什么？
7. 失败时下一条最有区分度的证据是什么？
8. 哪些结论仍需要真实重启或业务验证？

学习提示：每次验证都先说明证据属于当前状态、持久配置、`unit` 状态还是功能结果；不能用其中一层替代其他层。

目录存在，不等于文件系统已经挂载；当前已经挂载，也不等于重启后仍会出现；Swap 已经带有签名，也不等于内核正在使用它。存储题最常见的失误不是某个选项记错，而是拿一层证据替另一层下结论：看到 `/srv/project` 能写文件便认为挂载完成，看到 `/etc/fstab` 有一行便认为启动安全，看到 `free -h` 有 Swap 总量便认为新增区域已经启用。

本章围绕两条状态链推进。文件系统侧是“源对象 → 当前挂载 → 挂载点可见内容 → `fstab` → `systemd mount unit` → 启动重建”；Swap 侧是“块对象 → Swap 签名 → 当前启用 → 优先级 → `fstab` → `systemd swap unit`”。前一章已经负责文件系统创建、UUID 与容量管理；本章只消费已确认的文件系统和块对象。NFS 的协议与权限语义留给第 26 章，`autofs` 的按需触发留给第 27 章。

概念

当前挂载 (active mount) 是某个 `mount namespace` 中已经建立的“文件系统源 - 目录目标”关系。设备存在和目录存在都不能证明这条关系成立；必须从内核当前挂载表读取 `source`、`target`、类型和选项。手工 ``mount`` 通常只改变这一运行时层。

概念

持久挂载 (persistent mount) 是系统能够在下一次启动或明确重入时重新建立目标关系的配置状态。RHEL 9 中常见真源是 ``/etc/fstab``；当前已挂载并不会反向生成可靠的 `fstab` 条目，所以“现在可用”和“下次启动仍成立”必须分别验证。

概念

挂载点遮蔽 (mount-point shadowing) 指新文件系统挂到非空目录后，目录原有内容暂时不可见，但并未被删除。卸载后原内容会重新出现。它既可能造成“数据消失”的误判，也可能掩盖挂载失败期间写入父文件系统的数据。

概念

`fstab` 条目是由六个字段组成的持久声明，分别描述源、目标、类型、选项、`dump` 字段和启动检查序列。它表达管理员意图，不是当前挂载事实；文本存在也不等于条目可解析、源可用或启动一定成功。

概念

`systemd mount unit` 是 `systemd` 对一个挂载点的运行对象。`fstab` 条目通常经生成器转换为临时 ``mount`` 或 ``swap`` `unit`；`unit` 名由路径转义得到，状态和 `Journal` 能解释启动阶段发生了什么，但生成 `unit` 不是应直接编辑的配置真源。

概念

Swap 当前状态与持久配置 至少分三层：`mkswap` 写入 Swap 签名，`swapon` 让内核当前启用该区域，fstab 决定后续启动或 `swapon -a` 是否重新启用。`USED=0` 仍可能是已启用状态；具体设备与优先级应从 `swapon --show` 读取。

操作语义先理解命令作用对象，再记关键形式。以下入口覆盖当前挂载、证据查询、busy 调查、fstab 验证与 Swap 生命周期。

mount / umount

SYNOPSIS

```
mount [-t TYPE] [-o OPTIONS] SOURCE TARGET
mount TARGET
umount TARGET
```

建立或解除当前 mount namespace 中的挂载关系；它们本身不会自动创建或删除 fstab 持久条目。

重要参数 / 形式

mount SOURCE TARGET

显式建立当前关系。

mount TARGET

从 fstab 补齐源、类型与选项，适合单条重入验证。

-t TYPE

显式指定文件系统类型。

-o ro,rw,nodev,nosuid,noexec

为当前挂载选择行为或限制。

umount TARGET

解除当前关系；busy 时先调查引用，不默认使用 lazy 或 force。

findmnt

SYNOPSIS

```
findmnt [--mountpoint DIR] [--source SOURCE]
findmnt --target PATH
findmnt --fstab [--mountpoint DIR]
findmnt --verify
```

从当前挂载表或配置表读取 source、target、类型、选项和层次关系，是本章的主证据入口。

重要参数 / 形式

`--mountpoint DIR / -M DIR`

只匹配这个目录本身作为挂载点的记录。

`--target PATH / -T PATH`

查询任意路径实际位于哪个已挂载文件系统；路径不是独立挂载点时会返回其父文件系统。

`--source SOURCE`

查找某个源当前挂到哪里。

`-R TARGET`

查看目标下的嵌套挂载。

`--fstab`

读取 fstab 视图，而不是当前关系。

`--verify`

静态核对 fstab；不能替代实际挂载或真实重启。

fuser / lsof

SYNOPSIS

```
fuser -vm TARGET
lsof TARGET
lsof +D DIRECTORY
```

在 `umount` 报 busy 时识别持有工作目录、打开文件或其他引用的进程，再选择最小释放方式。

重要参数 / 形式

`fuser -m`

把目标按已挂载文件系统解释。

`-v`

显示用户、PID 和访问类型。

`lsof TARGET`

查询明确路径的打开对象。

`lsof +D DIRECTORY`

递归扫描目录树；范围大时成本较高，不应无条件使用。

fstab 编辑与重入验证

SYNOPSIS

```
systemctl daemon-reload
findmnt --verify
mount -a
```

fstab 修改后，先让 systemd 重新加载生成器结果，再做静态核对和当前环境中的实际重入测试。

重要参数 / 形式

`daemon-reload`

重新读取 unit 与 generator 结果；不等于挂载成功。

`findmnt --verify`

发现字段、类型和部分引用问题。

`mount -a`

尝试 fstab 中未挂载且非 `noauto` 的文件系统；已挂载目标可能不会被重新测试。

真实重启

仍是启动链最终验证之一，但必须在静态检查、重入测试和恢复条件充分后执行。

`mkswap` / `swapon` / `swapoff`

SYNOPSIS

```
mkswap [-L LABEL] DEVICE
swapon [-p PRIORITY] DEVICE
swapoff DEVICE
swapon -a
```

分别写入 Swap 签名、当前启用、当前停用，以及按 fstab 重新启用自动条目；每一步改变的状态层不同。

重要参数 / 形式

`mkswap -L LABEL`

写入签名和可读标签；这是覆盖性操作，必须先确认设备身份。

`swapon -p N`

以显式优先级启用，数值越高越优先。

`swapoff DEVICE`

只停用目标区域；先检查 USED、内存余量和其他 Swap。

`swapon -a`

依据 fstab 启用自动 Swap 条目；不应与危险的 `swapoff -a` 混淆。

`swapon --show` / `systemd unit`

SYNOPSIS

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
systemctl status srv-project.mount
systemctl status dev-vdb2.swap
```

读取具体 Swap 的当前状态与优先级，并从 ``.mount` / `.swap` unit` 观察 systemd 当前结果与失败入口。

重要参数 / 形式

`NAME, TYPE, SIZE, USED, PRIO`

设备级验收需要的核心列。

`systemd-escape --path --suffix=mount PATH`

把挂载路径转换为对应 unit 名。

`systemctl status UNIT`

查看 active/result 与近期日志；active 仍不能证明业务数据正确。

知识专题挂载不是复制：源、目标、当前关系与可见内容

第一次接触挂载时，人们容易把挂载点理解为“存储设备对应的文件夹”。这个比喻只能帮助入门，不能支撑排错。准确模型是：文件系统已经存在于某个源对象上，`mount` 把它的根目录接入当前目录树的某个目标目录；进程随后通过路径访问该文件系统。判断时必须分别识别源、目标、当前关系和目录原有内容。

① 知识点 source、target 和 active mount 是三个不同对象

- **source**：承载文件系统的对象，可以写成设备路径、`UUID=`、`LABEL=`，也可能是网络源；
- **target**：目录树中的挂载点，必须是目录；
- **active mount**：当前 mount namespace 中已经建立的源—目标关系。

设备存在只能证明源对象存在；目录存在只能证明目标目录存在。只有 `findmnt`、`mount` 的当前表或内核挂载信息才能证明 active mount。

```
findmnt --mountpoint /srv/project
findmnt -T /srv/project/subdir/file
```

第一条只匹配 `/srv/project` 本身作为挂载点的记录；第二条回答“给定路径实际位于哪个已挂载文件系统上”。如果 `/srv/project` 只是根文件系统中的普通目录，`findmnt -T` 仍会返回根文件系统，因此验收时还要比较期望的 source 和 target，不能只看命令有输出。

② 知识点当前挂载关系属于运行时状态

手工执行：

```
mount UUID=<REAL_UUID> /srv/project
```

只改变当前运行状态。系统重启后，内核重新建立根文件系统和启动过程要求的挂载，手工关系不会凭空恢复。要形成启动持久性，必须有可被启动系统消费的配置，RHEL 9 常见入口是 `/etc/fstab`。

在容器、隔离服务或使用不同 mount namespace 的进程中，某个挂载关系还可能只对特定 namespace 可见。本章不展开 namespace 管理命令，但保留一个判断边界：不要假设“一个终端能看到”就代表所有隔离进程都

能看到。

③知识点挂载到非空目录会遮蔽，不会删除

假设 `/srv/project` 原本包含 `local.txt`，挂载另一个文件系统后，路径 `/srv/project` 显示的是新文件系统的内容，`local.txt` 暂时不可见。正常卸载后，原内容重新出现。

这会制造两类高频事故：

1. 误判数据丢失：挂载后原目录项“消失”，其实被遮蔽；
2. 写入错误层：本应挂载的文件系统未挂载，应用继续向普通目录写数据；稍后挂载成功，这些数据又被遮蔽。

变更前应检查挂载点是否已有需要保留的内容：

```
findmnt --mountpoint /srv/project  
ls -la /srv/project
```

若目录非空，先确定这些内容属于预期文件系统还是父文件系统，不能直接覆盖现场。

④验证点“路径可写”不是挂载成功的证据

`touch /srv/project/test` 成功只证明当前路径可写；它可能仍位于根文件系统。更可靠的证据链是：

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project  
df -hT /srv/project
```

`findmnt` 核对 source、target、类型和当前选项，`df` 提供该路径所在文件系统的容量视角。功能写入测试应放在身份核对之后。

Cheatsheet

source 是文件系统来源，target 是目录，active mount 是当前关系；目录存在 ≠ 已挂载；挂载会遮蔽原内容；先用 `findmnt` 证明身份，再做读写测试。

操作专题用 findmnt 建立可信的挂载证据

`mount` 不带参数可以列出当前挂载，但输出通常很长，容易让目标关系淹没在伪文件系统和系统挂载中。

`findmnt` 更适合围绕“目标路径、源对象、配置表或层次关系”定向查询。本专题把它作为本章主证据入口，而不是只在操作结束时随手运行一次。

①操作点按挂载点查询当前关系

```
findmnt --mountpoint /srv/project  
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
```

第一条保留表头，便于阅读；第二条适合任务验收或脚本化采集。检查时至少回答：

- `SOURCE` 是否为目标文件系统；
- `TARGET` 是否正好是题目要求的目录；
- `FSTYPE` 是否符合要求；
- `OPTIONS` 中是否存在目标选项和意外限制。

②操作点查询任意路径实际落在哪个文件系统

```
findmnt -T /srv/project/reports/summary.txt
```

`-T / --target` 的路径解析适合调查“应用实际写入了哪个文件系统”。当挂载缺失时，它可能返回 `/` 对应的文件系统，这正是区别“目录存在”和“目标挂载存在”的证据。

③操作点按源查询并识别同源多挂载

```
findmnt --source UUID=<REAL_UUID>
findmnt --source /dev/mapper/projectvg-projectlv
```

一个源可能通过 `bind mount` 或其他方式出现在多个目标。按源查询可以发现“文件系统确实挂载了，但不在题目要求的位置”。设备路径可能存在多种别名，必要时结合 `lsblk -f`、`blkid` 和 `findmnt` 输出交叉核对。

④操作点查看子挂载和层次关系

```
findmnt -R /srv/project
```

递归视图对卸载诊断很重要。即使没有进程打开普通文件，目标下方的子挂载仍可能使整体卸载流程失败或留下意外状态。先从层次关系判断，再决定是否逐个处理子挂载。

⑤比较点`findmnt`、`df`、`lsblk`和`mountpoint`的证据边界

命令	主要回答	不能单独证明
<code>findmnt</code>	当前挂载源、目标、类型、选项、层次	<code>fstab</code> 一定正确；业务数据一定正确
<code>df -hT PATH</code>	路径所在文件系统的容量和使用率	期望 <code>source</code> ；持久配置
<code>lsblk -f</code>	块设备、文件系统签名、UUID、部分挂载点	当前挂载的全部层次与选项
<code>mountpoint -q DIR</code>	目录是否是一个挂载点	挂载源、类型和选项是否正确

精确挂载点用 ``findmnt --mountpoint``；任意路径归属用 ``findmnt -T``；同源挂载用 ``--source``；busy 前先 ``findmnt -R``；``df`` 是容量证据，不是持久证据。

操作专题建立和调整当前挂载

当前挂载操作的重点不是记忆一种固定命令，而是明确 source 的表达式、target 是否安全、选项改变了哪些行为，以及本次操作是否只存在于当前运行状态。

①操作点使用设备路径、UUID 或 LABEL 挂载

基本形式：

```
mount <SOURCE> <TARGET>
```

常见示例：

```
mount /dev/vdb1 /srv/project
mount UUID=<REAL_UUID> /srv/project
mount LABEL=PROJECT /srv/project
```

设备路径直观，但枚举名或映射别名可能变化；UUID 通常最适合唯一标识文件系统；LABEL 可读性更强，但管理员需要保证不会出现歧义。UUID 和 LABEL 必须从当前对象查询，不能复制讲义示例。

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

文件系统创建、重新格式化和部分签名操作归第 23 章，本章只消费已经确认的身份。

②参数点 ``-t``、``-o`` 与只读挂载

```
mount -t xfs -o nodev,nosuid UUID=<REAL_UUID> /srv/project
mount -o ro UUID=<REAL_UUID> /srv/archive
```

- `-t` 显式指定文件系统类型；通常内核和工具可根据签名识别，但题目或异常场景可能要求明确类型；
- `-o` 传入逗号分隔选项；
- `ro` 建立只读挂载，`rw` 表示读写；
- `nodev`、`nosuid`、`noexec` 分别限制设备文件解释、setuid/setgid 语义和直接执行，不能互相替代。

`noexec` 不是完整安全沙箱，解释器仍可能读取脚本；这些选项应按目标使用，不应机械添加。

③操作点利用 fstab 简写建立当前挂载

当 `/etc/fstab` 已有唯一条目时，可只给 target 或 source：

```
mount /srv/project
mount UUID=<REAL_UUID>
```

此时 `mount` 从 `fstab` 补齐另一端、类型和选项。这种操作适合验证单条配置，但仍要用 `findmnt` 检查实际结果。

④操作点remount 只改变当前状态

```
mount -o remount,ro /srv/project
mount -o remount,rw /srv/project
```

`remount` 用于调整当前已挂载对象的部分选项。它不会自动修改 `/etc/fstab`。若题目要求重启后也保持只读，必须同步修改持久配置并重新验证。

⑤验证点挂载后按身份、策略和功能三层验收

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
df -hT /srv/project
touch /srv/project/.rh24-write-test
rm -f /srv/project/.rh24-write-test
```

只读目标不应执行写入测试来“证明失败”；应改为读取已知文件或创建符合任务要求的验收方式。功能测试失败时，先根据错误区分只读、权限、SELinux、容量或文件系统问题，不要立即重新格式化。

Cheatsheet

``mount SOURCE TARGET`` 建立当前关系；UUID 唯一性通常优于路径，LABEL 要防重复；``-o`` 改变行为；`remount` 不改 `fstab`；验收依次看 source/target/type/options、容量、功能。

诊断专题安全卸载：target is busy 是保护证据

卸载失败不是命令“不给面子”，而是内核在阻止仍被引用的文件系统突然消失。正确流程是识别引用者、释放最小范围的引用、再重试；`lazy` 或 `force` 只能作为理解边界，不能成为考试和工作中的默认答案。

①操作点`umount` 解除当前关系

```
umount /srv/project
```

通常优先使用明确的挂载点。卸载不会删除文件系统内容，也不会删除挂载点目录，更不会自动删除 `fstab` 条目。若持久条目仍存在，下一次 `mount -a` 或启动可能重新挂载。

②诊断点当前工作目录会造成 busy

Shell 位于挂载点内部时：

```
cd /srv/project
umount /srv/project
```

可能失败。先离开该文件系统：

```
cd /
umount /srv/project
```

服务进程也可能把该文件系统作为工作目录或根目录；不能只检查当前终端。

③诊断点打开文件、执行映像和映射对象会保持引用

```
fuser -vm /srv/project
```

`fuser -vm` 按文件系统显示相关进程、用户和访问类型，是 `busy` 的首选证据之一。如果系统安装了 `lsuf`，可先查询挂载点本身；只有在目录树规模可控时才使用递归扫描：

```
lsuf +f -- /srv/project
# 或在范围较小且可接受递归成本时
lsuf +D /srv/project
```

不要看到 PID 就直接 `kill -9`。先识别进程属于哪个服务、是否可以正常停止、是否正在写数据，再通过应用或 `systemctl` 释放引用。

④诊断点子挂载需要先处理

```
findmnt -R /srv/project
```

若 `/srv/project/cache` 还有独立挂载，应先理解依赖并从内到外卸载。不要使用递归卸载隐藏自己尚未识别的业务挂载。

⑤边界lazy 和 force 为什么不是第一选择

- `umount -l` 会立即从目录树分离，等引用释放后再清理；进程可能继续访问旧对象，现场更难理解；
- `umount -f` 主要面向特定不可达网络文件系统等情况，本地正常挂载不应靠 `force` 跳过调查。

考试中看到 `busy`，推荐链路是：

症状 → findmnt -R → fuser/lsof → 识别管理者 → 最小停止或离开目录 → umount → findmnt 再验证

⑥验证点卸载后证明关系消失而目录仍在

```
findmnt --mountpoint /srv/project
mountpoint /srv/project
ls -ld /srv/project
```

预期是 current mount 消失，挂载点目录仍存在。若目录原有内容重新出现，需要按遮蔽模型解释，不要把它当成卸载生成的新文件。

Cheatsheet

busy 先查子挂载，再查进程引用；当前 Shell 要离开目录；优先正常停止服务；`-l`/`-f` 不是通用修复；卸载后验证关系消失、目录保留。

知识专题fstab 六字段：把对象、接入点和启动策略写清楚

/etc/fstab 的价值不是“记住一行模板”，而是用六个字段声明一个可重复建立的存储状态。每个字段回答不同问题：挂载谁、挂到哪里、按什么类型解释、使用什么策略、是否参与 dump、启动检查顺序如何。字段写对只是起点，还需要 systemd 重新加载、静态核对和实际挂载。

①知识点六字段的顺序与责任

```
fs_spec fs_file fs_vfstype fs_mntops fs_freq fs_passno
```

fs_spec

指定源对象。可写设备路径、UUID=...、LABEL=... 等；普通本地文件系统通常优先使用从当前对象查询到的真实 UUID。

fs_file

指定接入目录树的位置，例如 /srv/project。Swap 不形成目录挂载，第二字段通常写 none。

fs_vfstype

指定解释类型，例如 xfs、ext4，或用于交换区域的 swap。

fs_mntops

指定逗号分隔策略，例如 defaults,nodev,nosuid 或 defaults,pri=30。

fs_freq

传统 dump 字段。RHCSA 常见本地挂载和 Swap 条目通常写 0。

fs_passno

指定启动检查序列；0 表示不参与该序列。不要把某个文件系统的示例值机械套到所有类型。

普通挂载骨架：

```
UUID=<REAL_UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

Swap 骨架：

```
UUID=<REAL_UUID> none swap defaults,pri=30 0 0
```

②比较点UUID、LABEL 与设备路径

- `UUID=`：通常唯一，设备枚举顺序变化时仍能匹配同一文件系统；重新格式化后 `UUID` 会改变；
- `LABEL=`：可读性好，适合明确命名；若多个对象使用相同 `LABEL`，解析会产生歧义；
- `/dev/vdb1`、`/dev/sdb1`：简单直观，但某些环境中枚举顺序可能变化；
- `/dev/mapper/...` 或 `/dev/<VG>/<LV>`：LVM 对象常用稳定映射，但 LVM 创建和扩容归第 25 章。

写入前必须查询当前对象：

```
blkid /dev/vdb1
lsblk -f /dev/vdb1
```

③知识点`defaults` 不是 “没有选项”

`defaults` 表示一组默认挂载行为，由 `mount` 和文件系统实现共同解释。它不会自动添加 `nodev`、`nosuid`、`noexec`，也不会表达“失败无所谓”。安全或启动策略必须显式写出。

常用通用选项：

选项	作用
<code>rw / ro</code>	读写或只读
<code>nodev</code>	不解释该文件系统上的块/字符设备文件
<code>nosuid</code>	忽略 <code>setuid/setgid</code> 位的提权语义
<code>noexec</code>	不允许从该挂载直接执行二进制；不是完整隔离
<code>noauto</code>	<code>mount -a</code> 和正常自动挂载流程跳过，需手动触发
<code>nofail</code>	不把该挂载作为达到相关启动 <code>target</code> 的强制要求
<code>_netdev</code>	强制把挂载按依赖网络的挂载处理

文件系统专用选项数量很多，本章只覆盖通用且高价值的判断。遇到陌生选项应查 `mount(8)` 和对应文件系统手册页。

④边界字段中的空格不能直接出现

`fstab` 以空白分隔字段。路径或标签中包含空格时，需要使用转义形式，例如 `\040`。引号不能自动消除 `fstab` 字段分隔语义。考试环境通常避免复杂路径，但工作中看到带空格条目时，应先确认转义而不是反复增加引号。

⑤边界`nofail` 不能修复错误配置

启动风险：`nofail` 改变 `systemd` 依赖和启动完成条件，使相关挂载失败时系统仍可能继续到达目标状态。它不会修复错误 `UUID`、错误类型、损坏文件系统或权限问题。必需业务盘若错误地使用 `nofail`，系统可能“正常启动”但应用把数据写进未挂载目录，风险反而更大。

Cheatsheet

六字段依次是 `source`、`target`、`type`、`options`、`dump`、`fsck pass`；`UUID` 要从当前对象查询；``defaults`` 不包含所有安全选项；``noauto`` 是不自动挂，``nofail`` 是失败不阻断 `target`；空格用 ``\040``。

操作专题从修改 `fstab` 到无重启完整验证

编辑 `fstab` 后立即重启，是把一个可控的配置错误升级为启动故障。更稳妥的流程是在当前会话中完成静态检查、实际建立状态、身份核对和功能验证。无重启验证不能百分之百替代真实启动，但可以提前发现绝大多数语法、身份和挂载错误。

①操作点变更前建立基线并保留可回退副本

```
cp -a /etc/fstab /etc/fstab.rh24-before
findmnt --mountpoint /srv/project
findmnt --fstab --mountpoint /srv/project
```

备份不是让系统自动回滚，而是保留变更前配置。还应记录目标当前是否已挂载，因为 `mount -a` 通常跳过已经挂载的文件系统；若目标原本已由手工命令挂载，单纯运行 `mount -a` 不能证明新 `fstab` 条目能独立建立关系。

②操作点写入真实身份和明确策略

```
UUID=<REAL_UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

编辑时避免重复条目。一个目标出现多行或一个 `source` 被意外声明到多个目标，会使后续行为和验收复杂化。写完后先读取对应行，确认没有把占位符写入真配置。

③操作点让 `systemd` 重新运行生成器

```
systemctl daemon-reload
```


`daemon-reload` 让 `systemd` 重新读取 `unit` 配置并重新运行生成器，包括由 `fstab` 产生的 `mount/swap unit`。它不会自动把所有新条目挂载，也不会证明配置正确。

④验证点`findmnt --verify` 做静态核对

```
findmnt --verify
```

它用于检查 `fstab` 的可解析性和部分明显问题。静态检查通过不能证明源设备当前可用、文件系统可挂载或业务权限正确；失败时应按具体报告修正，不要直接重启尝试。

⑤操作点用`mount -a` 实际尝试尚未挂载的条目

```
mount -a
```

`mount -a` 处理 `fstab` 中适用且未被 `noauto` 排除的挂载，并通常忽略已经挂载的对象。命令无输出只表示没有直接报告错误，不能证明目标 `source`、类型和选项满足题意。

为了验证单条新配置能从“未挂载”状态建立，推荐在确认安全后：

```
umount /srv/project  
mount /srv/project
```

或在目标本来未挂载时直接 `mount -a`。生产系统不能为了测试随意卸载业务盘，应选择维护窗口或真实重启验证。

⑥验证点当前、持久、unit 和功能四层收口

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project  
findmnt --fstab --mountpoint /srv/project  
df -hT /srv/project  
systemctl status srv-project.mount
```

然后按任务执行最小功能验证。四层分别回答：当前关系、持久声明、容量/类型视角、`systemd unit` 状态。任何一层都不能独自扩大为“启动后业务一定正常”。

Cheatsheet

备份和基线 → 写真实 `UUID` → ``daemon-reload`` → ``findmnt --verify`` → 从未挂载状态实际挂载 → ``findmnt`/`df`/`unit`/功能分层验收`；无重启验证不等于真实重启。

知识专题systemd mount unit、启动依赖与网络边界

RHEL 9 的启动不是按 `fstab` 从上到下机械执行命令。`systemd` 读取由生成器创建的 `unit`，结合依赖和排序启动挂载。理解 `unit` 名和失败入口，可以把“启动卡住”从猜测变成可查询对象。

①知识点挂载点路径决定`.mount` unit 名

`/srv/project` 对应：

```
systemd-escape --path --suffix=mount /srv/project
# srv-project.mount
```

根挂载对应特殊的 `-.mount`。unit 名必须与路径转义规则一致，不能随意起名。由 `fstab` 生成的 unit 可查询：

```
systemctl status srv-project.mount
systemctl show srv-project.mount -p ActiveState,SubState,Result,Where,What
systemctl cat srv-project.mount
```

生成结果通常位于 `/run/systemd/generator*` 等运行时目录，重载或重启会重建；管理员应修改 `fstab`，而不是编辑生成文件。

②知识点unit active 只证明当前挂载成功

`active (mounted)` 表明 `systemd` 当前认为挂载已建立。它不能证明：

- `source` 是题目要求的那个对象；
- 选项完全正确；
- 应用目录权限和 SELinux 正确；
- 下一次设备缺失时启动仍符合业务要求。

因此 unit 状态必须与 `findmnt` 和功能证据组合。

③诊断点mount unit 失败时的证据链

```
systemctl status srv-project.mount
journalctl -b -u srv-project.mount
findmnt --verify
blkid
```

常见假设包括：`source` 不存在、UUID 过期、挂载点缺失、类型错误、选项不支持、文件系统本身异常。下一条证据应区分这些假设，而不是立即 `mkfs`。

④边界`\_netdev` 解决分类和排序，不解决连通性

`systemd` 通常根据文件系统类型识别网络挂载。若底层依赖网络但类型本身看起来像本地文件系统，例如网络块设备上承载普通文件系统，可在 `fstab` 选项中使用 `_netdev` 强制按网络挂载处理。它影响与网络上线及 `remote-fs.target` 等相关的依赖和排序。

`_netdev` 不会配置 IP、DNS、认证或远端服务，也不会让不可达存储变得可达。NFS 语法、版本和身份映射归第 26 章。

⑤边界`nofail` 改变启动要求，应由业务重要性决定

非关键、可选设备有时适合 `nofail`；关键数据盘通常不应为了“让机器能启动”而静默跳过。决策前要回答：

- 缺失时应用是否会误写父文件系统；
- 是否有监控能发现挂载失败；
- 是否允许系统继续提供降级服务；
- 恢复入口归第 30 章还是本章当前可修复。

Cheatsheet

路径用 ``systemd-escape`` 转 ``mount`` 名；generated unit 不直接编辑；失败查 `status` + 本次启动 Journal；``_netdev`` 是网络分类，``nofail`` 是启动依赖策略，都不是错误配置修复器。

操作专题Swap 的签名、当前启用和持久配置

Swap 不是普通文件系统挂载。它没有目录挂载点，内核把它作为换页后备区域使用。考试任务经常要求“新增 Swap、立即启用、重启后仍有效、不得影响现有 Swap”，这实际上要求分别处理签名、当前表、fstab 和设备级验收。

①操作点变更前记录现有 Swap

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
cat /proc/swaps
free -h
```

`swapon --show` 是设备级证据；`free -h` 只提供汇总。记录基线是为了证明新增操作没有停掉或覆盖已有 Swap。

②操作点`mkswap` 写入 Swap 签名

对已经由前置章节确认可写、无需保留数据的对象：

```
mkswap -L EXAMSWAP /dev/vdb2
blkid /dev/vdb2
```

`mkswap` 会写入新的 Swap 元数据和 UUID。它不是无损“标记命令”，不得对未知数据对象试运行。只存在 `TYPE="swap"` 仍不能证明当前启用。

③操作点`swapon` 当前启用并设置优先级

```
swapon -p 30 /dev/vdb2
```

或先写入 fstab 再执行：

```
swapon -a
```

`swapon -a` 根据 fstab 启用适用的 Swap 条目，通常跳过 `noauto`。当前命令成功不会自动写入 fstab。

④配置点按真实 UUID 持久启用

```
UUID=<REAL_SWAP_UUID> none swap defaults,pri=30 0 0
```

Swap 的第二字段通常写 `none`，第三字段为 `swap`。最后两个字段通常为 `0 0`。修改后：

```
systemctl daemon-reload  
findmnt --verify
```

Swap unit 名通常由设备身份和生成器决定，实际验收以 `swapon --show`、fstab 和必要的 `systemctl` 查询组合为主。

⑤验证点`USED=0` 仍可能是正常活动状态

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

只要设备出现在活动表中，它就是已启用 Swap；`USED=0` 只是当前没有页面被换出到该区域。不要为了制造使用量而人为施加内存压力。

⑥验证点从停用状态重新启用验证持久条目

在确认目标无关键 Swap 使用、系统内存充足且只操作新增区域后：

```
swapoff /dev/vdb2  
swapon -a  
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

这一过程验证 fstab 能重新启用该区域，但仍不是完整真实重启。禁止使用 `swapoff -a`，因为题目通常要求保留现有 Swap，且大范围换入可能导致内存压力或失败。

Cheatsheet

基线 → ``mkswap`` 写签名 → ``blkid`` 取 UUID → ``swapon -p`` 当前启用 → fstab ``none swap pri=`` → ``daemon-reload`/`verify`` → ``swapon --show``；``USED=0`` 不等于未启用。

知识专题Swap 优先级和安全停用

多个 Swap 区域可以同时活动。优先级让内核在多个后备区域之间做选择；停用则要求把该区域上的页面迁回 RAM 或其他 Swap。两者都需要设备级证据，不能只看总量。

①知识点数值更高的优先级更先使用

fstab 中使用：

```
UUID=<UUID_A> none swap defaults,pri=10 0 0
UUID=<UUID_B> none swap defaults,pri=30 0 0
```

当前启用也可用：

```
swapon -p 30 /dev/vdb2
```

实际优先级以 `swapon --show ... PRI0` 为准。不要只凭 fstab 文本判断，因为当前区域可能是在修改前以其他优先级启用的。

②知识点显式优先级比依赖默认分配更可验证

未显式设置时，优先级由系统分配，可能不符合题目要求。考试要求具体优先级时，应写入 `pri=<N>` 并重新启用目标 Swap，确认当前 PRI0。

③边界`swapoff` 前检查目标、使用量和可用内存

```
swapon --show --output NAME,SIZE,USED,PRI0
free -h
```

若目标 Swap 有大量使用且 RAM/其他 Swap 不足，`swapoff` 可能失败或造成严重内存压力。生产系统应安排窗口；考试环境也应只停用明确的新区域，不做全局清理。

④操作点删除或替换 Swap 的正确顺序

真正删除一个 Swap 时，通常按：

```
确认目标和内存余量
→ swapoff 指定区域
→ 从 fstab 移除或修改条目
→ daemon-reload
→ 再处理签名、分区或 LV
```

擦除签名和分区属于第 22 章边界；本章只强调不能在仍活动时破坏底层对象。

⑤验证点新增任务必须证明旧 Swap 保持不变

操作前后保存 `swapon --show`，比较已有 NAME、SIZE 和 PRIO。只证明总量增加，不足以证明原有条目没有被意外停用或改变。

Cheatsheet

`pri` 数值越高越优先；修改 `fstab` 后要让目标重新启用才会反映当前 PRIO；`swapoff` 先查 USED 和内存；永远优先指定单个目标，不用 `swapoff -a` 作为默认步骤。

诊断专题从症状推进到挂载、fstab、unit 或 Swap 层

存储诊断最危险的动作是“看到失败就重建”。错误 UUID、挂载缺失、unit 失败和 Swap 未启用都不要要求重新格式化。诊断要沿状态层逐步缩小，而不是破坏上游对象来制造一个新状态。

①诊断点目录存在，但数据写进了根文件系统

症状： `/srv/project` 可写，但容量和预期数据不对。

```
findmnt --mountpoint /srv/project
findmnt -T /srv/project
```

若 `--mountpoint` 没有记录，或 `-T` 显示它属于 `/`，说明目标文件系统当前未接入。下一步查看 `fstab`、`unit` 和 `source`，不要删除目录或重新 `mkfs`。

②诊断点`mount -a` 报 special device/UUID 不存在

```
blkid
lsblk -f
findmnt --fstab
```

区分：抄错 UUID、对象被重新格式化后 UUID 变化、设备尚未出现、配置使用了错误路径。修正 `fstab` 后 `daemon-reload`、`verify`、再挂载。重新格式化只会再次改变 UUID 并可能毁掉数据。

③诊断点wrong fs type、bad option 或 bad superblock

这类错误只给出了可能原因的集合。继续核对：

```
blkid <DEVICE>
findmnt --fstab --mountpoint /srv/project
journalctl -b -u srv-project.mount
```

可能是 `fstab` 类型错误、选项不支持、文件系统工具缺失或文件系统异常。文件系统检查与修复归第 23 章，不在未确认前执行破坏性命令。

④诊断点fstab 正确但 unit 仍 failed

```
systemctl status srv-project.mount
systemctl reset-failed srv-project.mount
systemctl start srv-project.mount
journalctl -b -u srv-project.mount
```

`reset-failed` 只清理失败状态计数，不修复根因。修正配置后再启动并核对 `findmnt`。

⑤诊断点挂载当前成功，但重启后缺失

检查：

```
findmnt --fstab --mountpoint /srv/project
systemctl cat srv-project.mount
systemctl is-enabled srv-project.mount
```

由 `fstab` 生成的 `mount unit` 不应机械套用 `service` 的 `enable` 逻辑；重点是条目是否存在、是否被 `noauto` 排除、`generator` 是否加载、启动依赖是否正确。真实重启仍需后续环境验证。

⑥诊断点Swap 有签名但总量没有增加

```
blkid /dev/vdb2
swapon --show
swapon /dev/vdb2
```

`blkid` 只证明签名。若 `swapon` 失败，读取具体错误并检查对象是否已被占用、是否重复启用或底层不合适。不要再次 `mkswap` 作为第一修复。

⑦诊断点fstab 中有 `pri=30`，当前 PRIO 却不同

配置只影响下一次启用。只针对该区域安全执行：

```
swapoff /dev/vdb2
swapon -a
swapon --show --output NAME,PRI0
```

若无法停用，先处理内存余量，不要扩大到 `swapoff -a`。

⑧诊断点使用 `nofail` 后系统启动但业务异常

检查目标是否真正挂载：

```
findmnt --mountpoint /srv/project
systemctl status srv-project.mount
journalctl -b -u srv-project.mount
```

`nofail` 可能让启动继续，却不会让应用自动停止向父文件系统写入。最小修复是恢复目标挂载并评估错误层数据，而不是继续忽略失败。

Cheatsheet

路径异常先 `findmnt`；UUID 错先查当前签名；unit failed 看本次启动 Journal；Swap 签名和活动表分开；配置改了但当前没变，考虑重新建立当前状态；任何诊断都不以重新格式化开场。

操作专题把考试步骤迁移成可审计的变更

真实系统往往已有 `fstab`、已有 Swap、已有应用数据和自动启动依赖。把本章迁移到工作中，需要在命令之外保留基线、变更范围、回退条件和分层证据。

①工作迁移变更前保存“对象与关系”而非只备份文件

除了备份 `fstab`，还应记录：

```
findmnt --mountpoint /srv/project
findmnt --fstab --mountpoint /srv/project
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
blkid /dev/vdb1 /dev/vdb2
```

这样才能回答变更前后发生了什么，而不是只有两个文本文件可比。

②工作迁移区分即时验证、重入验证和重启验证

- 即时验证：操作后立即查看当前状态；
- 重入验证：从未挂载/未启用状态用 `fstab` 再建立；
- 重启验证：真实启动完成后核对 unit、当前关系和业务功能。

本会话只能提供前两层的推荐命令，不能声称已完成第三层。

③工作迁移为监控选择可判定证据

对关键挂载，监控不应只检查目录存在。至少应检查期望 `target` 的 `source`、文件系统类型或标识，并监控容量。对 Swap，应记录活动区域和优先级，而不是只看总量。

④工作迁移为自动化准备稳定终态

手工终态应明确为：

- 真实 source 身份
- + 明确 target
- + 明确 fstype/options
- + 当前关系
- + 持久条目
- + unit/启动预期
- + 功能证据

RHCE 自动化章节再把该终态映射到 `ansible.posix.mount` 等模块。本章不展开 YAML，但要求状态模型足够稳定，避免自动化只“写一行 fstab”却不验证当前功能。

Cheatsheet

备份配置也记录当前关系；即时/重入/重启三种验证分开；监控检查身份而非目录；自动化消费终态，不替代本章对象模型。

经典任务为已有 XFS 建立安全的持久挂载

服务器中 `/dev/vdb1` 已由前置任务创建为 XFS 文件系统，但当前未挂载。`/srv/project` 已存在且为空，`/etc/fstab` 没有该对象的条目。要求：

- 使用当前系统查询到的真实 UUID；
- 持久挂载到 `/srv/project`；
- 挂载选项为 `defaults,nodev,nosuid`；
- 不重启完成当前和持久验证；
- 不重新格式化，不使用示例 UUID，不默认添加 `nofail`；
- 验收必须证明 source、target、类型、选项、容量视角和写入功能。

请先独立完成。参考解答从下一页开始。

参考解答从未挂载状态完成 fstab 与当前功能闭环

①调查确认源、挂载点和当前基线

```
lsblk -f /dev/vdb1
blkid /dev/vdb1
findmnt --mountpoint /srv/project
ls -la /srv/project
findmnt --fstab --mountpoint /srv/project
```

预期：`/dev/vdb1` 的 `TYPE` 为 `xfs`，可以读到真实 `UUID`；目标没有精确的当前挂载记录；目录没有需要保留的内容；`fstab` 没有冲突条目。任一条件不符合都先停止写配置。

②操作备份并写入真实条目

```
cp -a /etc/fstab /etc/fstab.rh24-c01-before
vi /etc/fstab
```

加入：

```
UUID=<从 blkid 获取的真实 UUID> /srv/project xfs defaults,nodev,nosuid 0 0
```

占位文本不能保留在真实文件中。

③验证重载、静态核对和实际挂载

```
systemctl daemon-reload
findmnt --verify
mount /srv/project
```

因为目标在操作前未挂载，`mount /srv/project` 会按 `fstab` 补齐 `source`、类型和选项，直接测试该条配置。

④分层验收当前关系、容量、unit 和功能

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
df -hT /srv/project
systemctl status srv-project.mount
touch /srv/project/.rh24-test
rm -f /srv/project/.rh24-test
```

检查 `OPTIONS` 中至少存在目标限制；输出中还会包含内核和文件系统添加的其他当前选项，不要求与 `fstab` 文本逐字相同。

⑤重入验证证明能从未挂载状态重建

在确认没有进程使用测试挂载后：

```
cd /
umount /srv/project
findmnt --mountpoint /srv/project
mount -a
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
```

这验证 `fstab` 在当前系统中可以重新建立状态。真实重启仍列入后续 live test。

⑥典型错误常见错误与修正方向

- 只执行 `touch`，实际写入根文件系统；
- 复制讲义 `UUID`；
- 目标已经手工挂载时直接运行 `mount -a`，误以为新条目被测试；
- 使用 `nofail` 掩盖错误 `UUID`；
- 忘记检查非空目录导致数据遮蔽。

经典任务新增 Swap、设置优先级并保持现有区域不变

系统已经存在至少一个活动 Swap。`/dev/vdb2` 是由前置任务确认可写的空闲分区，当前没有 Swap 签名。要求：

- 在 `/dev/vdb2` 上创建 Swap，LABEL 为 `EXAMSWAP`；
- 当前启用，优先级为 `30`；
- 使用真实 UUID 写入 `/etc/fstab`，启动时自动启用；
- 不得停用、删除或修改已有 Swap；
- 不得使用 `swapoff -a`；
- 使用设备级输出证明新区域、优先级和原有区域都正确；
- 在安全前提下，从停用状态用持久配置重新启用新区域。

请先独立完成。参考解答从下一页开始。

参考解答保留基线并完成 Swap 三层状态

①调查保存已有 Swap 和设备状态

```
swapon --show --output NAME,TYPE,SIZE,USED,PRI0 | tee /root/rh24-swap-before.txt
free -h
lsblk -f /dev/vdb2
blkid /dev/vdb2
```

若 `/dev/vdb2` 已有未知签名或归属，不执行 `mkswap`。空闲性调查的完整方法归第 22 章。

②操作写入 Swap 签名并获取真实 UUID

```
mkswap -L EXAMSWAP /dev/vdb2
blkid /dev/vdb2
```

记录输出中的 Swap UUID。

③操作当前按优先级启用

```
swapon -p 30 /dev/vdb2
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

确认新区域出现且 `PRI0` 为 30；原有 Swap 仍存在。

④配置写入持久条目

```
cp -a /etc/fstab /etc/fstab.rh24-c02-before
vi /etc/fstab
```

加入：

```
UUID=<真实 Swap UUID> none swap defaults,pri=30 0 0
```

然后：

```
systemctl daemon-reload
findmnt --verify
```

⑤重入验证只停用新增区域，再从 fstab 恢复

先检查新区域 `USED`、系统可用内存和其他 Swap；满足安全条件后：

```
swapoff /dev/vdb2
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
swapon -a
swapon --show --output NAME,TYPE,SIZE,USED,PRI0 | tee /root/rh24-swap-after.txt
```

禁止 `swapoff -a`。比较 before/after，确认原有区域没有消失或改变，新区域重新出现且优先级为 30。

⑥典型错误常见错误与修正方向

- `mkswap` 后只看 `blkid`，没有 `swapon`；
- 只运行 `free -h`，无法证明具体设备和优先级；
- `fstab` 写入 `pri=30` 后不重新启用，当前 `PRI0` 仍是旧值；
- 为了验证执行 `swapoff -a`，破坏现有 Swap；
- `USED=0` 被误判为未启用。

本章收束任何持久存储任务都要同时回答“现在”和“下次启动”

挂载与 Swap 的共同方法是把配置和运行状态拆开：

```
对象身份
→ 当前关系或活动状态
→ 当前策略
→ fstab 持久声明
→ systemd 生成 unit
→ 实际重入或启动
→ 功能与数据证据
```

文件系统侧，`findmnt` 是当前关系的主证据；`fstab` 和生成 `unit` 说明持久意图；`df` 和读写验证补充容量与功能。Swap 侧，`blkid` 证明签名，`swapon --show` 证明当前活动和优先级，`fstab` 证明启动策略。`mount -a`、`swapon -a` 和 `findmnt --verify` 都是验证工具，但都不能独立替代真实重启和业务验收。

遇到故障时，不重新创建对象来“试一试”。先确定失败位于身份、当前关系、配置、生成 `unit`、启动依赖还是功能层；选择下一条最有区分度的证据，完成最小修复，再从对应层向下重新验收。

最终 Cheatsheet

```
# 当前挂载
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS --mountpoint /srv/project
findmnt -T /srv/project/path
findmnt -R /srv/project

# 当前操作
mount UUID=<UUID> /srv/project
umount /srv/project
fuser -vm /srv/project

# 持久配置验证
systemctl daemon-reload
findmnt --verify
mount -a
systemctl status srv-project.mount

# Swap
mkswap -L EXAMSWAP /dev/vdb2
swapon -p 30 /dev/vdb2
swapoff /dev/vdb2
swapon -a
swapon --show --output NAME,TYPE,SIZE,USED,PRI0
```

主要判断表

看到的证据	可以证明	仍不能证明
blkid 显示 UUID / TYPE=swap	对象具有相应签名	已挂载、已启用或会在启动时恢复
findmnt 命中目标	当前 mount namespace 存在挂载关系	/etc/fstab 正确、重启后仍成立
/etc/fstab 存在条目	管理员声明了持久策略	条目可解析、源可用、当前已经生效
findmnt --verify 无致命问题	fstab 结构和部分引用通过静态核对	mount -a 一定成功、真实重启一定安全
mount -a 返回成功	当前环境能处理尚未挂载的自动条目	已经挂载的目标条目被重新测试、业务数据正确
swapon --show 出现设备	Swap 当前已启用并可读取优先级	fstab 持久配置正确、重启后必然恢复
.mount / .swap unit 为 active	systemd 当前认为该 unit 成功	应用正在访问正确数据、启动链没有被 nofail 掩盖

向下一章交接

本章把“已有文件系统或 Swap 如何进入当前状态并形成启动持久性”讲完整。下一章《LVM 逻辑存储》将向下进入 PV、VG、LV 和容量变更；那里创建出的 LV 仍需回到本章模型，分别完成文件系统、当前挂载、fstab 与重入验证。不要把 lvcreate 成功扩大成“存储已经可用”，也不要再在 LVM 章节重复本章全部持久挂载机制。